

第5章 C++与STL入门

学习目标

- ☑ 熟悉 C++ 版算法竞赛程序框架
- ☑ 理解变量引用的原理
- ☑ 熟练掌握 string 与 stringstream
- ☑ 熟练掌握 C++ 结构体的定义和使用，包括构造函数和静态成员变量
- ☑ 了解常见的可重载运算符，包括四则运算、赋值、流式输入输出、()和[]
- ☑ 了解模板函数和模板类的概念
- ☑ 熟练掌握 STL 中排序和检索的相关函数
- ☑ 熟练掌握 STL 中 vector、set 和 map 这 3 个容器
- ☑ 了解 STL 中的集合相关函数
- ☑ 理解栈、队列和优先队列的概念，并能用 STL 实现它们
- ☑ 熟练掌握随机数生成方法，并能结合 assert 宏进行测试
- ☑ 能独立编写大整数类 BigInteger

在前 4 章中介绍了 C 语言的主要内容，已经足以应付许多算法竞赛的题目了。然而，“能写”并不代表“好写”，有些题目虽然可以用 C 语言写出来，但是用 C++ 写起来往往会更快，而且更不容易出错，所以在讨论算法之前，有必要对 C++ 进行一番讲解。

本章采用“实用主义”的写法，并不会对所有内容加以解释，但是这并不影响读者“依葫芦画瓢”。不过有时读者还是希望能更细致、准确地学习到相关知识。推荐读者在手边放一本 C++ 的参考读物，如 C++ 之父 Bjarne Stroustrup 的经典著作《C++ 程序设计语言》。尽管如此，本章的作用也不容忽视：C++ 是一门庞大的语言，大多数语言特性和库函数在算法竞赛中都是用不到（或者可以避开）的。而且算法竞赛有它自身的特点，即使对于资深 C++ 程序员来说，如果缺乏算法竞赛的经验，也很难总结出一套适用于算法竞赛的知识点和实践指南。因此，即使你已经很熟悉 C++ 语言，但笔者仍建议花一些时间浏览本章的内容，相信会有新的收获。

5.1 从 C 到 C++

C 语言是一门很有用的语言，但在算法竞赛中却不流行，原因在于它太底层，缺少一些“实用的东西”。例如，在 2013 年 ACM/ICPC 世界总决赛中，有 1347 份用 C++ 提交，323 份用 Java 提交，但一份用 C 提交的都没有。

既然如此，为什么还要花这么多篇幅介绍 C 语言呢？答案是 C++ 太复杂了。与其把 C++

学得一知半解，还不如先把 C 语言的基础打好。前面已经提到过，前 4 章的所有代码都可以直接作为 C++ 程序进行编译，所以请把前 4 章内容看作语言的核心部分，而把本章内容看作是可选的工具。如果某些工具难以掌握，索性避开就是了。

C++ 博大精深，但也有很多人诟病的地方。好在算法竞赛中的大多数选手只会用到其中很少的特性，本章的任务就是把这些特性介绍给读者，以供选用。

提示 5-1: C++ 的精华与糟粕并存。本章介绍的 C++ 特性是算法竞赛中最常用的部分，虽然不是解题所必需的，但值得学习。

5.1.1 C++ 版框架

虽然前面介绍的内容都可以直接用在 C++ 程序里，但有些并不是 C++ 的推荐写法，只是为了更好地兼容 C 语言才如此编写的。下面是 C++ 版的“a+b 程序”：

```
#include<cstdio>
int main() {
    int a, b;
    while(scanf("%d%d", &a, &b) == 2) printf("%d\n", a+b);
    return 0;
}
```

和之前的 C 程序比较，唯一的区别是 `stdio.h` 变成了 `cstdio`。事实上，`stdio.h` 仍然存在，但是 C++ 中推荐的头文件是 `cstdio`。类似地，`string.h` 变成了 `cstring`，`math.h` 变成了 `cmath`，`ctype.h` 变成了 `cctype`。带 `.h` 后缀的头文件依然存在，但并不被 C++ 所推荐使用。

提示 5-2: C++ 能编译大多数 C 语言程序。虽然 C 语言中大多数头文件在 C++ 中仍然可以使用，但推荐的方法是在 C 头文件之前加一个小写的 `c` 字母，然后去掉 `.h` 后缀。

下面是一个稍微复杂一点的程序，它展示了更多的常用 C++ 特性。

```
#include<iostream>
#include<algorithm>
using namespace std;
const int maxn = 100 + 10;
int A[maxn];
int main() {
    long long a, b;
    while(cin >> a >> b) {
        cout << min(a,b) << "\n";
    }
    return 0;
}
```

这次的变化就大多了，新增的两个头文件不再是以字符 `c` 开头的。有人会猜这一定是 C++ 特有的头文件——的确如此。`iostream` 提供了输入输出流，而 `algorithm` 提供了一些常用

算法，例如代码中的 `min`^①。`cin >> a` 的含义是从标注输入中读取 `a`，它的返回值是一个“已经读取了 `a` 的新流”，然后从这个新流中继续读取 `b`。如果流已经读完，`while` 循环将退出。这种方式相比 `scanf` 的最大优势就是不再需要记忆 `%d`、`%s` 等占位符，同时也避开了前面提到的“`long long` 类型的输入输出占位符不统一”的问题。当然，C++流也不是完美的，其最大缺点就是运行太慢，以至于很多竞赛题目会在题面中的显著位置注明：本题的输入量很大，请不要使用 C++ 的流输入^②。

还有一个新内容：`using namespace std`。这是什么意思呢？C++ 中有一个“名称空间”（`namespace`）的概念，用来缓解复杂程序的组织问题。例如张三写了一个函数叫 `my_good_function`（意思是“我的优秀函数”），李四也写了这样一个函数，但作用和张三的不同。如果有一天需要把他们的程序合在一起用，就会出问题：函数不能重名。虽然后面会讲到 C++ 支持函数重载，但如果这两个函数的参数类型也完全相同，则是不能重载的。一个解决方案是分别把函数写在各自的名称空间里，然后就可以用 `zhang3:my_good_function()` 和 `li4:my_good_function()` 这样的方式进行调用了。

基于这样的考虑，头文件 `iostream` 和 `algorithm` 里定义的内容放在 `std` 名称空间里。如果代码和该名称空间里的内容不重名，就可以用 `using namespace std` 的方法把 `std` 里的名字导入默认空间^③。这样就可以用 `cin` 代替 `std::cin`，`cout` 代替 `std::cout`，`min` 代替 `std::min` 了。不信的话，你可以把这行语句注释掉，再编译一次试试。

提示 5-3：C++ 中可以使用流简化输入输出操作。标准输入输出流在头文件 `iostream` 中定义，存在于名称空间 `std` 中。如果使用了 `using namespace std` 语句，则可以直接使用。

最后还有一个细节：声明数组时，数组大小可以使用 `const` 声明的常数（这在 C99 中是不允许的）。在 C++ 中，这种写法更为推荐，因此本书后面的代码中一律采用这样的写法，而不是用 `#define` 声明常数。

顺便一提，C++ 中的数据类型和 C 语言很接近，最显著的区别是多了一个 `bool` 来表示布尔值，然后用 `true` 和 `false` 分别表示真和假。虽然仍然可以用 `int` 来表示真假，但是用 `bool` 可以让程序更清晰。

5.1.2 引用

第 4 章中曾经介绍过交换两个变量的方法，最后给出的例子用到了指针，看上去不太自然。C++ 提供了“引用”，虽然在功能上比指针弱，但是减少了出错的可能，提高了代码的可读性。使用引用交换两个变量的方法如下：

```
#include<iostream>
using namespace std;

void swap2(int& a, int& b) {
```

^① C 语言里连 `min` 函数都没有，可想而知还有多少常用的东西是无法直接用的。

^② 不过流也可以加速，方法是关闭和 `stdio` 的同步，即调用 `ios::sync_with_stdio(false)`。

^③ 在工程上不推荐这样做，不过因为算法竞赛的程序通常很小（多数不到 200 行），所以这样做也无大碍。

```

    int t = a; a = b; b = t;
}

int main() {
    int a = 3, b = 4;
    swap2(a, b);
    cout << a << " " << b << "\n";
    return 0;
}

```

是不是很自然？如果在参数名之前加一个“&”符号，就表示这个参数按照传引用（by reference）的方式传递，而不是 C 语言里的传值（by value）方式传递。这样，在函数内改变参数的值，也会修改到函数的实参。按照第 4 章介绍的方法进行 gdb 调试，用 b swap2 加一个断点，然后用 r 命令执行，如下所示：

```

Breakpoint 1, swap2 (a=@0x22ff4c: 3, b=@0x22ff48: 4) at swap2.cpp:5
5 int t = a; a = b; b = t;
(gdb) bt
#0 swap2 (a=@0x22ff4c: 3, b=@0x22ff48: 4) at swap2.cpp:5
#1 0x004013e1 in main() at swap2.cpp:10
(gdb) up
#1 0x004013e1 in main() at swap2.cpp:10
10 swap2(a, b);
(gdb) print &a
$1 = (int *) 0x22ff4c
(gdb) print &b
$2 = (int *) 0x22ff48

```

看到了吗？main 函数里的变量 a、b 的地址和 swap2 执行时参数 a、b 引用的地址一样，实际上是“同一个东西”。

提示 5-4：C++ 中的引用就是变量的“别名”，它可以在一定程度上代替 C 中的指针。例如，可以用“传引用”的方式让函数内直接修改实参。

细心的读者可能注意到了，为什么函数叫 swap2 而不是 swap 呢？因为 algorithm 这个头文件里已经提供过了 swap，可以直接使用。这个 swap 比此处所写的 swap2 强太多了：它不仅同时支持 int、double 等所有内置类型，甚至还支持用户自己编写的结构体。它是如何做到这点的呢？我们很快就要学习到。

5.1.3 字符串

还记得前面所说的“数组不是一等公民”吗？C 语言中的字符串就是字符数组，所以也不是“一等公民”，处处受限。例如，如何编写一个函数，把两个字符串拼接成一个长字

字符串？这个任务看上去简单，实际上却暗藏陷阱：新字符串的存储空间从哪里来？从第4章最后的讨论中可以知道：不能在函数中定义一个数组然后返回它的地址，因为函数返回后其中局部变量的地址便失效了。因此“字符串拼接”函数必须申请新的内存空间以存放结果，用完之后还要将申请的空间“退回去”，这会很麻烦。另外，字符串数组本身并不保存字符串长度，每次需要时都要用 `strlen` 函数重算一次。如果字符串很长，则 `strlen` 函数的开销将不容忽视^①。为了避免不必要的 `strlen` 调用，可以在某个变量中保存字符串的长度，但这样一来，程序会变得更加复杂，难以调试。总而言之，C 语言处理字符串并不方便。

C++ 提供了一个新的 `string` 类型，用来替代 C 语言中的字符数组。用户仍然可以继续用字符数组当字符串用，但是如果希望程序更加简单、自然，`string` 类型往往是更好的选择。例如，C++ 的 `cin/cout` 可以直接读写 `string` 类型，却不能读写字符数组；`string` 类型还可以像整数那样“相加”，而在 C 语言里只能使用 `strcat` 函数。

提示 5-5：C++ 在 `string` 头文件里定义了 `string` 类型，直接支持流式读写。`string` 有很多方便的函数和运算符，但速度有些慢。

考虑这样一个题目：输入数据的每行包含若干个（至少一个）以空格隔开的整数，输出每行中所有整数之和。如果只能使用字符与字符数组，一般有两种方案：一是使用 `getchar()` 边读边算，代码较短，但容易写错，并且相对较难理解^②；二是每次读取一行，然后再扫描该行的字符，同时计算结果。如果使用 C++，代码可以很简单。

```
#include<iostream>
#include<string>
#include<sstream>
using namespace std;

int main() {
    string line;
    while(getline(cin, line)) {
        int sum = 0, x;
        stringstream ss(line);
        while(ss >> x) sum += x;
        cout << sum << "\n";
    }
    return 0;
}
```

`string` 类在 `string` 头文件中，而 `stringstream` 在 `sstream` 头文件中。首先用 `getline` 函数读一行数据（相当于 C 语言中的 `fgets`，但由于使用 `string` 类，无须指定字符串的最大长度），然后用这一行创建一个“字符串流”——`ss`。接下来只需像读取 `cin` 那样读取 `ss` 即可。

^① 如果已完成了第3章的思考题，相信对此深有感触。

^② 有些选手非常习惯这种思维方式，但是根据笔者的经验，也有很多选手非常不习惯这种思维方式。

提示 5-6: 可以把 string 作为流进行读写, 定义在 sstream 头文件中。

虽然 string 和 sstream 都很方便, 但 string 很慢, sstream 更慢, 应谨慎使用^①。

5.1.4 再谈结构体

C++除了支持结构体 struct 之外, 还支持类 class。C++不再需要用 typedef 的方式定义一个 struct, 而且在 struct 里除了可以有变量(称为成员变量)之外还可以有函数(称为成员函数)。在工程中, 一般用 struct 定义“纯数据”的类型, 只包含较少的辅助成员函数, 而用 class 定义“拥有复杂行为”的类型, 不过为了简单起见, 本书中只使用 struct 而不使用 class。另外, “成员变量”、“成员函数”、“构造函数”等很多 C++ struct 里新加的概念同样适用于 class^②, 所以不用担心在本章中学到的内容为“非主流”。

提示 5-7: C++中的结构体除了可以拥有成员变量(用 a.x 的方式访问)之外, 还可以拥有成员函数(用 a.add(1,2)的方式访问)。为了简单起见, 本书中只使用 struct 而不使用 class, 但 struct 的很多概念和写法同样适用于 class。

下面是一个例子:

```
#include<iostream>
using namespace std;

struct Point {
    int x, y;
    Point(int x=0, int y=0):x(x),y(y) {}
};

Point operator + (const Point& A, const Point& B) {
    return Point(A.x+B.x, A.y+B.y);
}

ostream& operator << (ostream &out, const Point& p) {
    out << "(" << p.x << ", " << p.y << ")";
    return out;
}

int main() {
    Point a, b(1,2);
    a.x = 3;
    cout << a+b << "\n";
}
```

^① 具体有多慢? 试试就知道了。请读者自行编写程序测试。

^② 事实上, 在 C++中 struct 和 class 最主要的区别是默认访问权限和继承方式不同, 而其他方面的差异很小。

```
return 0;
}
```

上面的代码多数可以“望文知义”。结构体 `Point` 中定义了一个函数，函数名也叫 `Point`，但是没有返回值。这样的函数称为构造函数（ctor）。构造函数是在声明变量时调用的，例如，声明 `Point a, b(1,2)` 时，分别调用了 `Point()` 和 `Point(1,2)`。注意这个构造函数的两个参数后面都有“=0”字样，其中 0 为默认值。也就是说，如果没有指明这两个参数的值，就按 0 处理，因此 `Point()` 相当于 `Point(0,0)`。“`:x(x),y(y)`”则是一个简单的写法，表示“把成员变量 `x` 初始化为参数 `x`，成员变量 `y` 初始化为参数 `y`”。也可以写成：

```
Point(int x=0, int y=0) { this->x = x; this->y = y; }
```

这里的“`this`”是指向当前对象的指针。`this->x` 的意思是“当前对象的成员变量 `x`”，即 `(*this).x`。

提示 5-8： C++ 中的结构体可以有一个或多个构造函数，在声明变量时调用。

提示 5-9： C++ 中的函数（不只是构造函数）参数可以拥有默认值。

提示 5-10： 在 C++ 结构体的成员函数中，`this` 是指向当前对象的指针。

接下来为这个结构体定义了“加法”，并且在实现中用到构造函数。这样，就可以用 `a+b` 的形式计算两个结构体 `a` 和 `b` 的“和”了。

最后，定义这个结构体的流输出方式，然后就可以用 `cout << p` 来输出一个 `Point` 结构体 `p` 了。

5.1.5 模板

回顾第 4 章中介绍过的 `sum` 函数：

```
int sum(int* begin, int* end) {
    int *p = begin;
    int ans = 0;
    for(int *p = begin; p != end; p++)
        ans += *p;
    return ans;
}
```

这个函数没有错误，但比较局限——只能求整数数组的和，不能求 `double` 数组的和，更不能求 `Point` 数组的和。没关系，可以把这个函数改一下。

```
template<typename T>
T sum(T* begin, T* end) {
    T *p = begin;
    T ans = 0;
    for(T *p = begin; p != end; p++)
        ans = ans + *p;
}
```

```
return ans;
```

```
    }
```

这样，就可以用 `sum` 函数给 `double` 数组和 `Point` 数组求和了。

```
int main() {
```

```
    double a[] = {1.1, 2.2, 3.3, 4.4};
```

```
    cout << sum(a, a+4) << "\n";
```

```
    Point b[] = { Point(1,2), Point(3,4), Point(5,6), Point(7,8) };
```

```
    cout << sum(b, b+4) << "\n";
```

```
    return 0;
```

```
}
```

细心的读者应该已发现了上述 `sum` 函数和第4章中写的有点不同：把“`ans += *p`”改成了“`ans = ans + *p`”。这样做的原因是 `Point` 结构体中只定义了“+”运算符，没有定义“+=”。结构体和类（`class`）也可以是带模板的。例如，上述 `Point` 结构体中的 `x` 和 `y` 是 `int` 型的，但有时需要的是 `double` 型的 `x` 和 `y`，“+”和“<<”的逻辑不变。可以用类似的写法把 `Point` 变成模板。

```
template <typename T>
```

```
struct Point {
```

```
    T x, y;
```

```
    Point(T x=0, T y=0):x(x),y(y) {}
```

```
};
```

然后把“+”和“<<”的代码也稍加改变：

```
template <typename T>
```

```
Point<T> operator + (const Point<T>& A, const Point<T>& B) {
```

```
    return Point<T>(A.x+B.x, A.y+B.y);
```

```
}
```

```
template <typename T>
```

```
ostream& operator << (ostream &out, const Point<T>& p) {
```

```
    out << "(" << p.x << ", " << p.y << ")";
```

```
    return out;
```

```
}
```

这样就可以同时使用 `int` 型和 `double` 型的 `Point` 了：

```
int main() {
```

```
    Point<int> a(1,2), b(3,4);
```

```
    Point<double> c(1.1,2.2), d(3.3,4.4);
```

```
    cout << a+b << " " << c+d << "\n";
```

```
    return 0;
```

```
}
```


虽然模板在工程中的应用范围很广，而且功能十分强大^①，但选手们却很少会在算法竞赛中亲自编写模板。那为什么还要介绍模板呢？主要是因为模板有助于读者更好地理解 STL。

5.2 STL 初步

STL 是指 C++ 的标准模板库（Standard Template Library）。它很好用，但也很复杂。本节将介绍 STL 中的一些常用算法和容器，在后面的章节中还会继续介绍本节没有涉及的其他内容。

5.2.1 排序与检索

例题 5-1 大理石在哪儿（Where is the Marble?, UVa 10474）

现有 N 个大理石，每个大理石上写了一个非负整数。首先把各数从小到大排序，然后回答 Q 个问题。每个问题问是否有一个大理石写着某个整数 x ，如果是，还要回答哪个大理石上写着 x 。排序后的大理石从左到右编号为 $1 \sim N$ 。（在样例中，为了节约篇幅，所有大理石上的数合并到一行，所有问题也合并到一行。）

样例输入：

```
4 1
2 3 5 1
5
5 2
1 3 3 3 1
2 3
```

样例输出：

```
CASE# 1:
5 found at 4
CASE# 2:
2 not found
3 found at 3
```

【分析】

题目意思已经清楚了：先排序，再查找。使用 `algorithm` 头文件中的 `sort` 和 `lower_bound` 很容易完成这两项操作，代码如下：

```
#include<cstdio>
#include<algorithm>
using namespace std;
```

^① 有兴趣的读者可以研究一下 C++ 的模板元编程（template metaprogramming）。在 boost 库中有很多模板元编程的优秀例子。

```

const int maxn = 10000;

int main() {
    int n, q, x, a[maxn], kase = 0;
    while(scanf("%d%d", &n, &q) == 2 && n) {
        printf("CASE# %d:\n", ++kase);
        for(int i = 0; i < n; i++) scanf("%d", &a[i]);
        sort(a, a+n); //排序
        while(q--) {
            scanf("%d", &x);
            int p = lower_bound(a, a+n, x) - a; //在已排序数组 a 中寻找 x
            if(a[p] == x) printf("%d found at %d\n", x, p+1);
            else printf("%d not found\n", x);
        }
    }
    return 0;
}

```

上面的代码比第4章中的排序代码简单很多，因为省略了一个 `compare` 函数——`sort` 使用数组元素默认的大小比较运算符进行排序，只有在需要按照特殊依据进行排序时才需要传入额外的比较函数。

另外，`sort` 可以对任意对象进行排序，不一定是内置类型。如果希望用 `sort` 排序，这个类型需要定义“小于”运算符，或者在排序时传入一个“小于”函数。排序对象可以存在于普通数组里，也可以存在于 `vector` 中（参见 5.2.2 节）。前者用 `sort(a, a+n)` 的方式调用，后者用 `sort(v.begin(), v.end())` 的方式调用。`lower_bound` 的作用是查找“大于或者等于 `x` 的第一个位置”。

为什么 `sort` 可以对任意对象进行排序呢？学习了前面内容，相信读者可以猜到，这是因为 `sort` 是一个模板函数。

提示 5-11: `algorithm` 头文件中的 `sort` 可以给任意对象排序，包括内置类型和自定义类型，前提是类型定义了“<”运算符。排序之后可以用 `lower_bound` 查找大于或等于 `x` 的第一个位置。待排序/查找的元素可以放在数组里，也可以放在 `vector` 里。

还有一个 `unique` 函数可以删除有序数组中的重复元素，后面的例题中将展示其用法。

5.2.2 不定长数组：vector

`vector` 就是一个不定长数组。不仅如此，它把一些常用操作“封装”在了 `vector` 类型内部。例如，若 `a` 是一个 `vector`，可以用 `a.size()` 读取它的大小，`a.resize()` 改变大小，`a.push_back()` 向尾部添加元素，`a.pop_back()` 删除最后一个元素。

`vector` 是一个模板类，所以需要 `vector<int> a` 或者 `vector<double> b` 这样的方式来声明一个 `vector`。`vector<int>` 是一个类似于 `int a[]` 的整数数组，而 `vector<string>` 就是一个类似

于 `string a[]` 的字符串数组。`vector` 看上去像是“一等公民”，因为它们可以直接赋值，还可以作为函数的参数或者返回值，而无须像传递数组那样另外用一个变量指定元素个数。

例题 5-2 木块问题 (The Blocks Problem, UVa 101)

从左到右有 n 个木块，编号为 $0 \sim n-1$ ，要求模拟以下 4 种操作（下面的 a 和 b 都是木块编号）。

- `move a onto b`: 把 a 和 b 上方的木块全部归位，然后把 a 摆在 b 上面。
- `move a over b`: 把 a 上方的木块全部归位，然后把 a 放在 b 所在木块堆的顶部。
- `pile a onto b`: 把 b 上方的木块全部归位，然后把 a 及上面的木块整体摆在 b 上面。
- `pile a over b`: 把 a 及上面的木块整体摆在 b 所在木块堆的顶部。

遇到 `quit` 时终止一组数据。 a 和 b 在同一堆的指令是非法指令，应当忽略。

样例输入：

1234

样例输出：

1234 -> 3087 -> 8352 -> 6174 -> 6174

【分析】

每个木块堆的高度不确定，所以用 `vector` 来保存很合适；而木块堆的个数不超过 n ，所以用一个数组来存就可以了。代码如下：

```
#include <cstdio>
#include <string>
#include <vector>
#include <iostream>
using namespace std;

const int maxn = 30;
int n;
vector<int> pile[maxn]; //每个 pile[i] 是一个 vector

//找木块 a 所在的 pile 和 height，以引用的形式返回调用者
void find_block(int a, int& p, int& h) {
    for(p = 0; p < n; p++)
        for(h = 0; h < pile[p].size(); h++)
            if(pile[p][h] == a) return;
}

//把第 p 堆高度为 h 的木块上方的所有木块移回原位
void clear_above(int p, int h) {
    for(int i = h+1; i < pile[p].size(); i++) {
        int b = pile[p][i];
```

```

    pile[b].push_back(b); //把木块b放回原位
}
pile[p].resize(h+1); //pile只应保留下标 0~h 的元素
}

//把第 p 堆高度为 h 及其上方的木块整体移动到 p2 堆的顶部
void pile_onto(int p, int h, int p2) {
    for(int i = h; i < pile[p].size(); i++)
        pile[p2].push_back(pile[p][i]);
    pile[p].resize(h);
}

void print() {
    for(int i = 0; i < n; i++) {
        printf("%d:", i);
        for(int j = 0; j < pile[i].size(); j++) printf(" %d", pile[i][j]);
        printf("\n");
    }
}

int main() {
    int a, b;
    cin >> n;
    string s1, s2;
    for(int i = 0; i < n; i++) pile[i].push_back(i);
    while(cin >> s1 >> a >> s2 >> b) {
        int pa, pb, ha, hb;
        find_block(a, pa, ha);
        find_block(b, pb, hb);
        if(pa == pb) continue; //非法指令
        if(s2 == "onto") clear_above(pb, hb);
        if(s1 == "move") clear_above(pa, ha);
        pile_onto(pa, ha, pb);
    }
    print();
    return 0;
}

```

数据结构的核心是 `vector<int> pile[maxn]`, 所有操作都是围绕它进行的。`vector` 就像一个二维数组, 只是第一维的大小是固定的 (不超过 `maxn`), 但第二维的大小不固定。上述代码还有一个值得学习的技巧: 输入一共有 4 种指令, 但如果完全独立地处理各指令, 代码就会变得冗长而且易错。更好的方法是提取出指令之间的共同点, 编写函数以减少重复代码。

提示 5-12: `vector` 头文件中的 `vector` 是一个不定长数组，可以用 `clear()` 清空，`resize()` 改变大小，用 `push_back()` 和 `pop_back()` 在尾部添加和删除元素，用 `empty()` 测试是否为空。`vector` 之间可以直接赋值或者作为函数的返回值，像是“一等公民”一样。

5.2.3 集合：set

集合与映射也是两个常用的容器。`set` 就是数学上的集合——每个元素最多只出现一次。和 `sort` 一样，自定义类型也可以构造 `set`，但同样必须定义“小于”运算符。

例题 5-3 安迪的第一个字典（Andy's First Dictionary, UVa 10815）

输入一个文本，找出所有不同的单词（连续的字母序列），按字典序从小到大输出。单词不区分大小写。

样例输入：

Adventures in Disneyland

Two blondes were going to Disneyland when they came to a fork in the road. The sign read: "Disneyland Left."

So they went home.

样例输出（为了节约篇幅只保留前 5 行）：

a
adventures
blondes
came
disneyland

【分析】

本题没有太多的技巧，只是为了展示 `set` 的用法：由于 `string` 已经定义了“小于”运算符，直接使用 `set` 保存单词集合即可。注意，输入时把所有非字母的字符变成空格，然后利用 `stringstream` 得到各个单词。

```
#include<iostream>
#include<string>
#include<set>
#include<sstream>
using namespace std;
```

```
set<string> dict; //string 集合
```

```
int main() {
    string s, buf;
    while(cin >> s) {
```

```

for(int i = 0; i < s.length(); i++)
    if(isalpha(s[i])) s[i] = tolower(s[i]); else s[i] = ' ';
stringstream ss(s);
while(ss >> buf) dict.insert(buf);
}
for(set<string>::iterator it = dict.begin(); it != dict.end(); ++it)
    cout << *it << "\n";
return 0;
}

```

上面的代码用到了 `set` 中元素已从小到大排好序这一性质，用一个 `for` 循环即可从小到大遍历所有元素。

代码里的 `set<string>::iterator` 是什么？`dict.begin()` 和 `dict.end()` 又是什么？`iterator` 的意思是迭代器，是 STL 中的重要概念，类似于指针。和“`vector` 类似于数组”一样，这里的“类似”指的是用法类似。还记得第 4 章中的那个 `sum` 函数吗？

```

int sum(int* begin, int* end) {
    int *p = begin;
    int ans = 0;
    for(int *p = begin; p != end; p++)
        ans += *p;
    return ans;
}

```

这个 `for` 循环是不是和上面的代码很像？实际上，上面参数中的 `begin` 和 `end` 就是仿照 STL 中的迭代器命名的。

5.2.4 映射：map

`map` 就是从键（key）到值（value）的映射。因为重载了 `[]` 运算符，`map` 像是数组的“高级版”。例如可以用一个 `map<string,int> month_name` 来表示“月份名字到月份编号”的映射，然后用 `month_name["July"] = 7` 这样的方式来赋值。

例题 5-4 反片语（Anagrams, UVa 156）

输入一些单词，找出所有满足如下条件的单词：该单词不能通过字母重排，得到输入文本中的另外一个单词。在判断是否满足条件时，字母不分大小写，但在输出时应保留输入中的大小写，按字典序进行排列（所有大写字母在所有小写字母的前面）。

样例输入：

```

ladder came tape soon leader acme RIDE lone Dreis peat
ScAlE orb eye Rides dealer NotE derail LaCeS drIed
noel dire Disk mace Rob dries
#

```

样例输出：

```
Disk
NotE
derail
drIed
eye
ladder
soon
```

【分析】

把每个单词“标准化”，即全部转化为小写字母后再进行排序，然后再放到 `map` 中进行统计。代码如下：

```
#include<iostream>
#include<string>
#include<cctype>
#include<vector>
#include<map>
#include<algorithm>
using namespace std;

map<string,int> cnt;
vector<string> words;

//将单词 s 进行“标准化”
string repr(const string& s) {
    string ans = s;
    for(int i = 0; i < ans.length(); i++)
        ans[i] = tolower(ans[i]);
    sort(ans.begin(), ans.end());
    return ans;
}

int main() {
    int n = 0;
    string s;
    while(cin >> s) {
        if(s[0] == '#') break;
        words.push_back(s);
        string r = repr(s);
        if(!cnt.count(r)) cnt[r] = 0;
        cnt[r]++;
    }
```

```

}
vector<string> ans;
for(int i = 0; i < words.size(); i++)
    if(cnt[repr(words[i])] == 1) ans.push_back(words[i]);
sort(ans.begin(), ans.end());
for(int i = 0; i < ans.size(); i++)
    cout << ans[i] << "\n";
return 0;
}

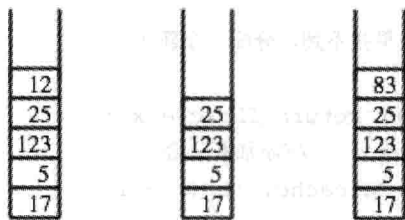
```

此例说明，如果没有良好的代码设计，是无法发挥 STL 的威力的。如果没有想到“标准化”这个思路，就很难用 map 简化代码。

提示 5-13: set 头文件中的 set 和 map 头文件中的 map 分别是集合与映射。二者都支持 insert、find、count 和 remove 操作，并且可以按照从小到大的顺序循环遍历其中的元素。map 还提供了“[]”运算符，使得 map 可以像数组一样使用。事实上，map 也称为“关联数组”。

5.2.5 栈、队列与优先队列

STL 还提供 3 种特殊的数据结构：栈、队列与优先队列。所谓栈，就是符合“后进先出”（Last In First Out, LIFO）规则的数据结构，有 PUSH 和 POP 两种操作，其中 PUSH 把元素压入“栈顶”，而 POP 从栈顶把元素“弹出”，如图 5-1 所示。



Original stack. After pop(). After push(83).

图 5-1 PUSH 和 POP 操作

讲一个有趣的笑话。如何判断一个人是不是程序员？答：问它 PUSH 的反义词是什么。回答 PULL 的是普通人，而回答 POP 的才是程序员^①。这个笑话间接地说明了“栈”这个数据结构在计算机中的重要性。

STL 的栈定义在头文件<stack>中，可以用“stack<int> s”方式声明一个栈。

提示 5-14: STL 的 stack 头文件提供了栈，用“stack<int> s”方式定义，用 push()和 pop()实现元素的入栈和出栈操作，top()取栈顶元素（但不删除）。

例题 5-5 集合栈计算机(The SetStack Computer, ACM/ICPC NWERC 2006, UVa12096)

有一个专门为了集合运算而设计的“集合栈”计算机。该机器有一个初始为空的栈，

^① 如果你想较真的话，这里有一个反例：经常使用 git 的程序员也有可能回答 pull。

并且支持以下操作。

- ❑ PUSH: 空集 “{}” 入栈。
 - ❑ DUP: 把当前栈顶元素复制一份后再入栈。
 - ❑ UNION: 出栈两个集合，然后把二者的并集入栈。
 - ❑ INTERSECT: 出栈两个集合，然后把二者的交集入栈。
 - ❑ ADD: 出栈两个集合，然后把先出栈的集合加入到后出栈的集合中，把结果入栈。
- 每次操作后，输出栈顶集合的大小（即元素个数）。例如，栈顶元素是 $A = \{\{\}, \{\{\}\}\}$ ，

下一个元素是 $B = \{\{\}, \{\{\{\}\}\}\}$ ，则：

- ❑ UNION 操作将得到 $\{\{\}, \{\{\}\}, \{\{\{\}\}\}\}$ ，输出 3。
- ❑ INTERSECT 操作将得到 $\{\{\}\}$ ，输出 1。
- ❑ ADD 操作将得到 $\{\{\}, \{\{\{\}\}\}, \{\{\}, \{\{\}\}\}\}$ ，输出 3。

输入不超过 2000 个操作，并且保证操作均能顺利进行（不需要对空栈执行出栈操作）。

【分析】

本题的集合并不是简单的整数集合或者字符串集合，而是集合的集合。为了方便起见，此处为每个不同的集合分配一个唯一的 ID，则每个集合都可以表示成所包含元素的 ID 集合，这样就可以用 STL 的 `set<int>` 来表示了，而整个栈则是一个 `stack<int>`。

```
typedef set<int> Set;
map<Set, int> IDcache;    //把集合映射成 ID
vector<Set> Setcache;    //根据 ID 取集合

//查找给定集合 x 的 ID。如果找不到，分配一个新 ID
int ID (Set x) {
    if (IDcache.count(x)) return IDcache[x];
    Setcache.push_back(x);    //添加新集合
    return IDcache[x] = Setcache.size() - 1;
}
```

对任意集合 s （类型是上面定义的 `Set`），`IDcache[s]` 就是它的 ID，而 `Setcache[IDcache[s]]` 就是 s 本身。下面的 `ALL` 和 `INS` 是两个宏^①：

```
#define ALL(x) x.begin(), x.end()
#define INS(x) inserter(x, x.begin())
```

分别表示“所有的内容”以及“插入迭代器”，具体作用可以从代码中推断出来，有兴趣的读者可以查阅 STL 文档以了解更详细的信息。主程序如下，请读者注意 STL 内置的集合操作（如 `set_union` 和 `set_intersection`）。

```
stack<int> s;    //题目中的栈
int n;
cin >> n;
```

^① 宏 (macro) 是一个很复杂的话题，这里读者暂时可以把带参数的宏理解为“类似于函数的东西”。

```

for(int i = 0; i < n; i++) {
    string op;
    cin >> op;
    if (op[0] == 'P') s.push(ID(Set()));
    else if (op[0] == 'D') s.push(s.top());
    else {
        Set x1 = Setcache[s.top()]; s.pop();
        Set x2 = Setcache[s.top()]; s.pop();
        Set x;
        if (op[0] == 'U') set_union (ALL(x1), ALL(x2), INS(x));
        if (op[0] == 'I') set_intersection (ALL(x1), ALL(x2), INS(x));
        if (op[0] == 'A') { x = x2; x.insert(ID(x1)); }
        s.push(ID(x));
    }
    cout << Setcache[s.top()].size() << endl;
}

```

本题极为重要，后面章节中有一些例题也使用了本题的解决方法，建议读者仔细体会。队列是符合“先进先出”（First In First Out, FIFO）原则的“公平队列”，无须过多介绍，如图 5-2 所示。

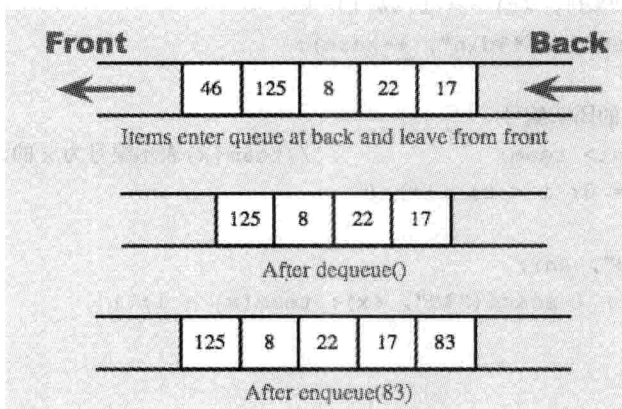


图 5-2 队列

STL 队列定义在头文件<queue>中，可以用“queue<int> s”方式声明一个队列。

提示 5-15: STL 的 queue 头文件提供了队列，用“queue<int> s”方式定义，用 push()和 pop()进行元素的入队和出队操作，front()取队首元素（但不删除）。

例题 5-6 团体队列（Team Queue, UVa 540）

有 t 个团队的人正在排一个长队。每次新来一个人时，如果他有队友在排队，那么这个新人会插队到最后一个队友的身后。如果没有任何一个队友排队，则他会排到长队的队尾。输入每个团队中所有队员的编号，要求支持如下 3 种指令（前两种指令可以穿插进行）。

- ENQUEUE x: 编号为 x 的人进入长队。
- DEQUEUE: 长队的队首出队。
- STOP: 停止模拟。

对于每个 DEQUEUE 指令, 输出出队的人的编号。

【分析】

本题有两个队列: 每个团队有一个队列, 而团队整体又形成一个队列。例如, 有 3 个团队 1, 2, 3, 队员集合分别为 {101, 102, 103, 104}、{201, 202} 和 {301, 302, 303}, 当前长队为 {301, 303, 103, 101, 102, 201}, 则 3 个团队的队列分别为 {103, 101, 102}、{201} 和 {301, 303}, 团队整体的队列为 {3, 1, 2}。代码如下:

```
#include<cstdio>
#include<queue>
#include<map>
using namespace std;

const int maxt = 1000 + 10;

int main() {
    int t, kase = 0;
    while(scanf("%d", &t) == 1 && t) {
        printf("Scenario #%d\n", ++kase);

        //记录所有人的团队编号
        map<int, int> team;           //team[x]表示编号为 x 的人所在的团队编号
        for(int i = 0; i < t; i++) {
            int n, x;
            scanf("%d", &n);
            while(n--) { scanf("%d", &x); team[x] = i; }
        }

        //模拟
        queue<int> q, q2[maxt];      //q 是团队的队列, 而 q2[i] 是团队 i 成员的队列
        for(;;) {
            int x;
            char cmd[10];
            scanf("%s", cmd);
            if(cmd[0] == 'S') break;
            else if(cmd[0] == 'D') {
                int t = q.front();
                printf("%d\n", q2[t].front()); q2[t].pop();
                if(q2[t].empty()) q.pop(); //团体 t 全体出队列
            }
        }
    }
}
```

```

else if(cmd[0] == 'E') {
    scanf("%d", &x);
    int t = team[x];
    if(q2[t].empty()) q.push(t); //团队 t 进入队列
    q2[t].push(x);
}
}
printf("\n");
return 0;
}

```

优先队列是一种抽象数据类型（Abstract Data Type, ADT），行为有些像队列，但先出队列的元素不是先进队列的元素，而是队列中优先级最高的元素，这样就可以允许类似于“急诊病人插队”这样的事情发生。

STL 的优先队列也定义在头文件<queue>里，用“priority_queue<int> pq”来声明。这个 pq 是一个“越小的整数优先级越低的优先队列”。由于出队元素并不是最先进队的元素，出队的方法由 queue 的 front()变为了 top()。

自定义类型也可以组成优先队列，但必须为每个元素定义一个优先级。这个优先级并不需要一个确定的数字，只需要能比较大小即可。看到这里，是不是想起了 sort？没错，只要元素定义了“小于”运算符，就可以使用优先队列。在一些特殊的情况下，需要使用自定义方式比较优先级，例如，要实现一个“个位数大的整数优先级反而小”的优先队列，可以定义一个结构体 cmp，重载“()”运算符，使其“看上去”像一个函数^①，然后用“priority_queue<int, vector<int>, cmp> pq”的方式定义。下面是这个 cmp 的定义：

```

struct cmp {
    bool operator() (const int a, const int b) const { //a 的优先级比 b 小时返回
true
        return a % 10 > b % 10;
    }
};

```

对于一些常见的优先队列，STL 提供了更为简单的定义方法，例如，“越小的整数优先级越大的优先队列”可以写成“priority_queue<int, vector<int>, greater<int>> pq”。注意，最后两个“>”符号不要写在一起，否则会被很多（但不是所有）编译器误认为是“>>”运算符。

提示 5-16: STL 的 queue 头文件提供了优先队列，用“priority_queue<int> s”方式定义，用 push()和 pop()进行元素的入队和出队操作，top()取队首元素（但不删除）。

^① 在 C++ 中，重载了“()”运算符的类或结构体叫做仿函数（functor）。

例题 5-7 丑数 (Ugly Numbers, UVa 136)

丑数是指不能被 2, 3, 5 以外的其他素数整除的数。把丑数从小到大排列起来，结果如下：

1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15, ...

求第 1500 个丑数。

【分析】

本题的实现方法有很多种，这里仅提供一种，即从小到大生成各个丑数。最小的丑数是 1，而对于任意丑数 x ， $2x$ 、 $3x$ 和 $5x$ 也都是丑数。这样，就可以用一个优先队列保存所有已生成的丑数，每次取出最小的丑数，生成 3 个新的丑数。唯一需要注意的是，同一个丑数有多种生成方式，所以需要判断一个丑数是否已经生成过。代码如下：

```
#include<iostream>
#include<vector>
#include<queue>
#include<set>
using namespace std;
typedef long long LL;
const int coeff[3] = {2, 3, 5};

int main() {
    priority_queue<LL, vector<LL>, greater<LL> > pq;
    set<LL> s;
    pq.push(1);
    s.insert(1);
    for(int i = 1; ; i++) {
        LL x = pq.top(); pq.pop();
        if(i == 1500) {
            cout << "The 1500'th ugly number is " << x << ".\n";
            break;
        }
        for(int j = 0; j < 3; j++) {
            LL x2 = x * coeff[j];
            if(!s.count(x2)) { s.insert(x2); pq.push(x2); }
        }
    }
    return 0;
}
```

答案：859963392。

5.2.6 测试 STL

和自己写的代码一样，库也是需要测试的。一方面是因为库也是人写的，也有可能

bug, 另一方面是因为测试之后能更好地了解库的用法和优缺点。

提示 5-17: 库不一定没有 bug, 使用之前测试库是一个好习惯。

测试的方法大同小异, 下面只以 `sort` 为例进行介绍。首先, 写一个简单的测试程序:

```
int a[] = {3,2,4};
sort(a, a+3);
printf("%d %d %d\n", a[0], a[1], a[2]);
```

输出为 2 3 4, 是一个令人满意的结果。但这样就够了吗? 不! 测试程序太简单, 说明不了问题。应该写一个更加通用的程序, 随机生成很多整数, 然后排序。

为了随机生成整数, 先来看看随机数发生器。核心函数是 `cstdlib` 中的 `rand()`, 它生成一个闭区间 $[0, \text{RAND_MAX}]$ 内的均匀随机整数 (均匀的含义是: 该区间内每个整数被随机获取的概率相同), 其中 `RAND_MAX` 至少为 32767 ($2^{15}-1$), 在不同环境下的值可能不同。严格地说, 这里的随机数是“伪随机数”, 因为它也是由数学公式计算出来的, 不过在算法领域, 多数情况下可以把它当作真正的随机数。

如何产生 $[0, n]$ 之间的整数呢? 很多人喜欢用 `rand()%n` 产生区间 $[0, n-1]$ 内的一个随机整数, 姑且不论这样产生的整数是否仍然分布均匀, 只要 n 大于 `RAND_MAX`, 此法就不能得到期望的结果。由于 `RAND_MAX` 很有可能只有 32767 这么小, 在使用此法时应当小心。另一个方法是执行 `rand()` 之后先除以 `RAND_MAX`, 得到 $[0, 1]$ 之间的随机实数, 扩大 n 倍后四舍五入, 得到 $[0, n]$ 之间的均匀整数。这样, 在 n 很大时“精度”不好 (好比把小图放大后会看到“锯齿”), 但对于普通的应用, 这样做已经可以满足要求了^①。

提示 5-18: `cstdlib` 中的 `rand()` 可生成闭区间 $[0, \text{RAND_MAX}]$ 内均匀分布的随机整数, 其中 `RAND_MAX` 至少为 32767。如果要生成更大的随机整数, 在精度要求不太高的情况下可以用 `rand()` 的结果“放大”得到。

需要随机数的程序在最开始时一般会执行一次 `srand(time(NULL))`, 目的是初始化“随机数种子”。简单地说, 种子是伪随机数计算的依据。种子相同, 计算出来的“随机数”序列总是相同。如果不调用 `srand` 而直接使用 `rand()`, 相当于调用过一次 `srand(1)`, 因此程序每次执行时, 将得到同一套随机数。

不要在同一个程序每次生成随机数之前都重新调用一次 `srand`。有的初学者抱怨“`rand()` 产生的随机数根本不随机, 每次都相同”, 就是因为误解了 `srand` 的作用。再次强调, 请只在程序开头调用一次 `srand`, 而不要在同一个程序中多次调用。

提示 5-19: 可以用 `cstdlib` 中的 `srand` 函数初始化随机数种子。如果需要程序每次执行时使用一个不同的种子, 可以用 `ctime` 中的 `time(NULL)` 为参数调用 `srand`。一般来说, 只在程序执行的开头调用一次 `srand`。

“同一套随机数”可能是好事也可能是坏事。例如, 若要反复测试程序对不同随机数

^① 如果坚持需要更高的精度, 可以采取多次随机的方法。

据的响应，需要每次得到的随机数不同。一个简单的方法是使用当前时间 `time(NULL)`（在 `ctime` 中）作为参数调用 `srand`。由于时间是不断变化的，每次运行时，一般会得到一套不同的随机数。之所以说“一般会”，是因为 `time` 函数返回的是自 UTC 时间 1970 年 1 月 1 日 0 点以来经过的“秒数”，因此每秒才变化一次。如果你的程序是由操作系统自动批量执行的，可能因为每次运行的间隔时间过短，导致在相邻若干次执行时 `time` 的返回值全部相同。一个解决办法是在测试程序的主函数中设置一个循环，做足够多次测试后再退出^①。

另一方面，如果发现某程序对于一组随机数据报错，就需要在调试时“重现”这组数据。这时，“每次相同的随机序列”就显得十分重要了。不同的编译器计算随机数的方法可能不同。如果是不同编译器编译出来的程序，即使是用相同的参数调用 `srand()`，也可能得到不同的随机序列。

讲了这么多，下面可以编写随机程序了：

```
void fill_random_int(vector<int>& v, int cnt) {
    v.clear();
    for(int i = 0; i < cnt; i++)
        v.push_back(rand());
}
```

注意 `srand` 函数是在主程序开始时调用，而不是每次测试时调用。参数是 `vector<int>` 的引用。为什么不把这个 `v` 作为返回值，而要写到参数里呢？答案是：避免不必要的值被复制。如果这样写：

```
vector<int> fill_random_int(int cnt) {
    vector<int> v;
    for(int i = 0; i < cnt; i++)
        v.push_back(rand());
    return v;
}
```

实际上函数内的局部变量 `v` 中的元素需要逐个复制给调用者。而用传引用的方式调用，就避免了这些复制过程。

提示 5-20：把 `vector` 作为参数或者返回值时，应尽量改成用引用方式传递参数，以避免不必要的值被复制。

这两个函数可以同时存在于一份代码中，因为 C++ 支持函数重载，即函数名相同但参数不同的两个函数可以同时存在。这样，编译器可以根据函数调用时参数类型的不同判断应该调用哪个函数。如果两个函数的参数相同^②只是返回值不同，是不能重载的。

提示 5-21：C++ 支持函数重载，但函数的参数类型必须不同（不能只有返回值类型不同）。

^① 还有一个更通用的方法将在附录 A 中说明。

^② 准确地说，应该是参数类型相同，参数的名字是无关紧要的。

写完了随机数发生器之后，就可以正式测试 `sort` 函数了，程序如下：

```
void test_sort(vector<int>& v) {
    sort(v.begin(), v.end());
    for(int i = 0; i < v.size()-1; i++)
        assert(v[i] <= v[i+1]);
}
```

新内容是上面的 `assert` 宏，其用法是“`assert(表达式)`”，作用是：当表达式为真时无变化，但当表达式为假时强行终止程序，并且给出错误提示。当然，上述程序也可以写成“`if(v[i] > v[i+1]) { printf("Error: v[i]>v[i+1]!\n"); abort(); }`”，但 `assert` 更简洁，而且可以知道是由代码中的哪一行引起的，所以在测试时常常使用它。

提示 5-22：测试时往往使用 `assert`。其用法是“`assert(表达式)`”，当表达式为假时强行终止程序，并给出错误提示。

和刚才一样，给参数 `v` 加上引用符的原因是为了避免 `vector` 复制，但函数执行完毕之后 `v` 会被 `sort` 改变。如果调用者不希望这个 `v` 被改变，就应该去掉“`&`”符号（即参数改成 `vector<int> v`），改回传值的方式。

下面是主程序，请注意 `srand` 函数的调用位置。顺便我们还测试了 `sort` 的时间效率，发现给 10^6 个整数排序几乎不需要时间。

```
int main() {
    vector<int> v;
    fill_random_int(v, 1000000);
    test_sort(v);
    return 0;
}
```

`vector`、`set` 和 `map` 都很快^①，其中 `vector` 的速度接近数组（但仍有差距），而 `set` 和 `map` 的速度也远远超过了“用一个 `vector` 保存所有值，然后逐个元素进行查找”时的速度。`set` 和 `map` 每次插入、查找和删除时间和元素个数的对数呈线性关系，其具体含义将在第 8 章中详细讨论。尽管如此，在一些对时间要求非常高的题目中，STL 有时会成为性能瓶颈，请读者注意。

5.3 应用：大整数类

在介绍 C 语言时，大家已经看到了很多整数溢出的情形。如果运算结果真的很大，就需要用到所谓的高精度算法，即用数组来储存整数，并模拟手算的方法进行四则运算。这些算法并不难实现，但是还应考虑一个易用性问题——如果能像使用 `int` 一样方便地使用大

^① 注意 `vector` 并不是所有操作都快。例如 `vector` 提供了 `push_front` 操作，但由于在 `vector` 首部插入元素会引起所有元素往后移动，实际上 `push_front` 是很慢的。

整数，那该有多好！相信读者已经想到解决方案了，那就是使用 `struct`。

5.3.1 大整数类 BigInteger

结构体 `BigInteger` 可用于储存高精度非负整数。

```
struct BigInteger {
    static const int BASE = 1000000000;
    static const int WIDTH = 8;
    vector<int> s;

    BigInteger(long long num = 0) { *this = num; } //构造函数
    BigInteger operator = (long long num) {      //赋值运算符
        s.clear();
        do {
            s.push_back(num % BASE);
            num /= BASE;
        } while(num > 0);
        return *this;
    }
    BigInteger operator = (const string& str) { //赋值运算符
        s.clear();
        int x, len = (str.length() - 1) / WIDTH + 1;
        for(int i = 0; i < len; i++) {
            int end = str.length() - i*WIDTH;
            int start = max(0, end - WIDTH);
            sscanf(str.substr(start, end-start).c_str(), "%d", &x);
            s.push_back(x);
        }
        return *this;
    }
};
```

其中，`s` 用来保存大整数的各个数位。例如，若是要表示 1234，则 `s={4,3,2,1}`。用 `vector` 而非数组保存数字的好处显而易见：不用关心这个整数到底有多大，`vector` 会自动根据情况申请和释放内存。

上面的代码中还有赋值运算符，有了它就可以用 `x = 123456789` 或者 `x = "1234567898765432123456789"` 这样的方式来给 `x` 赋值了。

提示 5-23：可以给结构体重载赋值运算符，使得用起来更方便。

之前已经介绍过 “<<” 运算符，类似的还有 “>>” 运算符，代码一并给出。

```
ostream& operator << (ostream &out, const BigInteger& x) {
    out << x.s.back();
```

```

for(int i = x.s.size()-2; i >= 0; i--) {
    char buf[20];
    sprintf(buf, "%08d", x.s[i]);
    for(int j = 0; j < strlen(buf); j++) out << buf[j];
}
return out;
}

istream& operator >> (istream &in, BigInteger& x) {
    string s;
    if(!(in >> s)) return in;
    x = s;
    return in;
}

```

这样，就可以用 `cin>>x` 和 `cout<<x` 的方式来进行输入输出了。怎么样，很方便吧？不仅如此，`stringstream` 也“自动”支持了 `BigInteger`，这得益于 C++ 中的类继承机制。简单地说^①，由于“>>”和“<<”运算符的参数是一般的 `istream` 和 `ostream` 类，作为“特殊情况”的 `cin/cout` 以及 `stringstream` 类型的流都能用上它。

上述代码中还有两点需要说明。一是 `static const int BASE = 100000000`，其作用是声明一个“属于 `BigInteger`”的常数。注意，这个常数不属于任何 `BigInteger` 类型的结构体变量，而是属于 `BigInteger` 这个“类型”的，因此称为静态成员变量，在声明时需要加 `static` 修饰符。在 `BigInteger` 的成员函数里可以直接使用这个常数（见上面的代码），但在其他地方使用时需要写成 `BigInteger::BASE`。

提示 5-24：可以给结构体声明一些属于该结构体类型的静态成员变量，方法是加上 `static` 修饰符。静态成员变量在结构体外部使用时要写成“结构体名::静态成员变量名”。

5.3.2 四则运算

这部分内容和 C++ 本身关系不大，但是由于高精度类非常常见，这里仍然给出代码（定义在结构体内部）：

```

BigInteger operator + (const BigInteger& b) const {
    BigInteger c;
    c.s.clear();
    for(int i = 0, g = 0; ; i++) {
        if(g == 0 && i >= s.size() && i >= b.s.size()) break;
        int x = g;
        if(i < s.size()) x += s[i];

```

^① 任何一本 C++ 语言教材都会介绍类继承，但它在算法竞赛中很少使用，所以这里略去细节。

```

        if(i < b.s.size()) x += b.s[i];
        c.s.push_back(x % BASE);
        g = x / BASE;
    }
    return c;
}

```

为了让使用更加简单（还记得之前为什么要修改 `sum` 函数吗？），还可以重新定义“+”运算符（定义在结构体内部）：

```

BigInteger operator += (const BigInteger& b) {
    *this = *this + b; return *this;
}

```

减法、乘法和除法的原理类似，这里不再赘述，请读者参考代码仓库。

5.3.3 比较运算符

下面实现“比较”操作（定义在结构体内部）：

```

bool operator < (const BigInteger& b) const {
    if(s.size() != b.s.size()) return s.size() < b.s.size();
    for(int i = s.size()-1; i >= 0; i--)
        if(s[i] != b.s[i]) return s[i] < b.s[i];
    return false; //相等
}

```

一开始就比较两个 `BigInteger` 的位数，如果不相等则直接返回，否则直接从后往前比较（因为低位在 `vector` 的前面）。注意，这样做的前提是两个数都没有前导零，否则，很可能出现“运算结果都没问题，但一比较就出错”的情况。

只需定义“小于”这一个符号，即可用它定义其他所有比较运算符（当然，对于 `BigInteger` 这个例子来说，“=”可以直接定义为 `s == b.s`，不过不具一般性）：

```

bool operator > (const BigInteger& b) const{ return b < *this; }
bool operator <= (const BigInteger& b) const{ return !(b < *this); }
bool operator >= (const BigInteger& b) const{ return !(*this < b); }
bool operator != (const BigInteger& b) const{ return b < *this || *this < b; }
bool operator == (const BigInteger& b) const{ return !(b < *this) && !(*this < b); }

```

可以同时用“<”和“>”把“!=”和“==”定义得更加简单，读者可以自行尝试。

还记得 `sort`、`set` 和 `map` 都依赖于类型的“小于”运算符吗？现在它们是不是已经自动支持 `BigInteger` 了？赶紧试试吧！

5.4 竞赛题目举例

例题 5-8 Unix ls 命令 (Unix ls, UVa 400)

输入正整数 n 以及 n 个文件名，排序后按列优先的方式左对齐输出。假设最长文件名有 M 字符，则最右列有 M 字符，其他列都是 $M+2$ 字符。

样例输入 (略，可以由样例输出推出)

样例输出：

```
-----
Alice      Chris      Jan         Marsha      Ruben
Bobby      Cindy      Jody        Mike         Shirley
Buffy      Danny      Keith       Mr._French  Sissy
Carol      Greg       Lori        Peter
```

【分析】

首先计算出 M 并算出行数，然后逐行逐列输出。代码如下：

```
#include<iostream>
#include<string>
#include<algorithm>
using namespace std;

const int maxcol = 60;
const int maxn = 100 + 5;
string filenames[maxn];

//输出字符串 s，长度不足 len 时补字符 extra
void print(const string& s, int len, char extra) {
    cout << s;
    for(int i = 0; i < len-s.length(); i++)
        cout << extra;
}

int main() {
    int n;
    while(cin >> n) {
        int M = 0;
        for(int i = 0; i < n; i++) {
            cin >> filenames[i];
            M = max(M, (int)filenames[i].length()); //STL 的 max
        }
        //计算列数 cols 和行数 rows
```

```

int cols = (maxcol - M) / (M + 2) + 1, rows = (n - 1) / cols + 1;
print("", 60, '-');
cout << "\n";
sort(filenamees, filenamees+n); //排序
for(int r = 0; r < rows; r++) {
    for(int c = 0; c < cols; c++) {
        int idx = c * rows + r;
        if(idx < n) print(filenamees[idx], c == cols-1 ? M : M+2, ' ');
    }
    cout << "\n";
}
return 0;
}

```

例题 5-9 数据库 (Database, ACM/ICPC NEERC 2009, UVa1592)

输入一个 n 行 m 列的数据库 ($1 \leq n \leq 10000$, $1 \leq m \leq 10$)，是否存在两个不同行 r_1, r_2 和两个不同列 c_1, c_2 ，使得这两行和这两列相同（即 (r_1, c_1) 和 (r_2, c_1) 相同， (r_1, c_2) 和 (r_2, c_2) 相同）。例如，对于如图 5-3 所示的数据库，第 2、3 行和第 2、3 列满足要求。

How to compete in ACM ICPC	Peter	peter@neerc.ifmo.ru
How to win ACM ICPC	Michael	michael@neerc.ifmo.ru
Notes from ACM ICPC champion	Michael	michael@neerc.ifmo.ru

图 5-3 数据库

【分析】

直接写一个四重循环枚举 r_1, r_2, c_1, c_2 可以吗？理论上可以，实际上却行不通。枚举量太大，程序会执行相当长的时间，最终获得 TLE（超时）。

解决方法是只枚举 c_1 和 c_2 ，然后从上到下扫描各行。每次碰到一个新的行 r ，把 c_1, c_2 两列的内容作为一个二元组存到一个 map 中。如果 map 的键值中已经存在这个二元组，该二元组映射到的就是所要求的 r_1 ，而当前行就是 r_2 。

这里有一个细节问题：如何表示由 c_1, c_2 两列组成的二元组？一种方法是直接用两个字符串拼成一个长字符串（中间用一个其他地方不可能出现的字符分隔），但是速度比较慢（因为在 map 中查找元素时需要进行字符串比较操作）。更值得推荐的方法是在主循环之前先做一个预处理——给所有字符串分配一个编号，则整个数据库中每个单元格都变成了整数，上述二元组就变成了两个整数。这个技巧已经在前面的例题“集合栈计算机”中用过，读者不妨再复习一下那道题目。

例题 5-10 PGA 巡回赛的奖金 (PGA Tour Prize Money, ACM/ICPC World Finals 1990, UVa207)

你的任务是为 PGA（美国职业高尔夫球协会）巡回赛计算奖金。巡回赛分为 4 轮，其中所有选手都能打前两轮（除非中途取消资格），得分相加（越少越好），前 70 名（包括

并列)晋级 (make the cut)。所有晋级选手再打两轮,前 70 名 (包括并列)有奖金。组委会事先会公布每个名次能拿的奖金比例。例如,若冠军比例是 18%,总奖金是\$1000000,则冠军奖金是\$180000。

输入保证冠军不会并列。如果第 k 名有 n 人并列,则第 $k \sim k+n-1$ 名的奖金比例相加后平均分给这 n 个人。奖金四舍五入到美分。所有业余选手不得奖金。例如,若业余选手得了第 3 名,则第 4 名会拿第 3 名的奖金比例。如果没取消资格的非业余选手小于 70 名,则剩下的奖金就不发了。

输入第一行为数据组数。每组数据前有一个空行,然后分为两部分。第一部分有 71 行 (各有一个实数),第一行为总奖金,第 $n+1$ 行为第 i 名的奖金比例。比例均保留 4 位小数,且总和为 100%。第 72 行为选手数 (最多 144),然后每行一个选手,格式为:

Player name RD1 RD2 RD3 RD4

业余选手名字后会有一“*”。犯规选手在犯规的那一轮成绩为 DQ,并且后面不再有其他成绩。但是只要没犯规,即使没有晋级,也会给出 4 轮成绩 (虽然在实际比赛中没晋级的选手只会有两个成绩)。输入保证至少有 70 个人晋级。

输入举例:

```
140
WALLY WEDGE      70 70 70 70
SANDY LIE        80 DQ
SID SHANKER*     90 99 62 61
...
JIMMY ABLE       69 73 80 DQ
```

输出应包含所有晋级到后半段 (make the cut) 的选手。输出信息包括:选手名字、排名、各轮得分、总得分以及奖金数。没有得奖则不输出,若有奖金,即使奖金是\$0.00 也要输出,保留两位小数)。如果此名次至少有两个人获得奖金,应在名次后面加“T”。犯规选手列在最后,总得分为 DQ,名次为空。如果有并列,则先按轮数排序,然后按各轮得分之和排序,最后按名字排序。两组数据的输出之间用一个空格隔开。

输出举例:

Player Name	Place	RD1	RD2	RD3	RD4	TOTAL	Money Won
WALLY WEDGE	1	70	70	70	70	280	\$180000.00
HENRY HACKER	2T	77	70	70	70	287	\$88000.00
TOMMY TWO IRON	2T	71	72	72	72	287	\$88000.00
BEN BIRDIE	4	70	74	72	72	288	\$48000.00
NORMAN NIBLICK*	4	72	72	72	72	288	
...							
LEE THREE WINES	71	99	99	99	98	395	\$2000.00
JOHNY MELAVO	72	99	99	99	99	396	
JIMMY ABLE		69	73	80		DQ	
EDDIE EAGLE		71	71			DQ	

【分析】

不难发现，第一个步骤是选出晋级选手，这涉及对所有选手“前两轮总得分”进行排序。接下来计算4轮总分，然后再排序一次，最后对排序结果依次输出。

输出过程不能大意：犯规选手要单独处理；在输出一行之前要先看看有没有并列的情况，如有则要一并处理（包括计算奖金平分情况）。本题没有技术上的难度，但比较考验选手的代码组织能力和对细节的处理，推荐读者一试。

例题 5-11 邮件传输代理的交互 (The Letter Carrier's Rounds, ACM/ICPC World Finals 1999, UVa814)

本题的任务为模拟发送邮件时 MTA（邮件传输代理）之间的交互。所谓 MTA，就是 email 地址格式 `user@mtaname` 的“后面部分”。当某人从 `user1@mta1` 发送给另一个人 `user2@mta2` 时，这两个 MTA 将会通信。如果两个收件人属于同一个 MTA，发送者的 MTA 只需与这个 MTA 通信一次就可以把邮件发送给这两个人。

输入每个 MTA 里的用户列表，对于每个发送请求（输入发送者和接收者列表），按顺序输出所有 MTA 之间的 SMTP（简单邮件协议）交互。协议细节参见原题。

发送人 MTA 连接收件人 MTA 的顺序应该与在输入中第一次出现的顺序一致。例如，若发件人是 `Hamdy@Cairo`，收件人列表为 `Conrado@MexicoCity`、`Shariff@SanFrancisco`、`Lisa@MexicoCity`，则 Cairo 应当依次连接 MexicoCity 和 SanFrancisco。

如果连接某个 MTA 之后发现所有收件人都不存在，则不应该发送 DATA。所有用户名均由不超过 15 个字母和数字组成。

【分析】

本题的关键是理清各个名词之间的逻辑关系以及把要做的事情分成几个步骤。首先是输入过程，把每个 MTA 里的用户列表保存下来。一种方法是用一个 `map<string, vector<string>>`，其中键是 MTA 名称，值是用户名列表。一个更简单的方法是用一个 `set<string>`，值就是邮件地址。

对于每个请求，首先读入发件人，分离出 MTA 和用户名，然后读入所有收件人，根据 MTA 出现的顺序进行保存，并且去掉重复。接下来读入邮件正文，最后按顺序依次连接每个 MTA，检查并输出每个收件人是否存在，如果至少有一个存在，则输出邮件正文。

本题的整个解决过程并不复杂，对于初学者来说是个不错的基础练习。参考代码如下：

```
#include<iostream>
#include<string>
#include<vector>
#include<set>
#include<map>
using namespace std;

void parse_address(const string& s, string& user, string& mta) {
    int k = s.find('@');
    user = s.substr(0, k);
    mta = s.substr(k+1);
}
```

```

int main() {
    int k;
    string s, t, user1, mta1, user2, mta2;
    set<string> addr;

    //输入所有 MTA, 转化为地址列表
    while(cin >> s && s != "") {
        cin >> s >> k;
        while(k--) { cin >> t; addr.insert(t + "@" + s); }
    }

    while(cin >> s && s != "") {
        parse_address(s, user1, mta1); //处理发件人地址

        vector<string> mta; //所有需要连接的 mta, 按照输入顺序
        map<string, vector<string> > dest; //每个 MTA 需要发送的用户
        set<string> vis;
        while(cin >> t && t != "") {
            parse_address(t, user2, mta2); //处理收件人地址
            if(vis.count(t)) continue; //重复的收件人
            vis.insert(t);
            if(!dest.count(mta2)) { mta.push_back(mta2); dest[mta2] = vector<string>(); }
            dest[mta2].push_back(t);
        }
        getline(cin, t); //把 "*" 这一行的回车吃掉

        //输入邮件正文
        string data;
        while(getline(cin, t) && t[0] != '*') data += " " + t + "\n";

        for(int i = 0; i < mta.size(); i++) {
            string mta2 = mta[i];
            vector<string> users = dest[mta2];
            cout << "Connection between " << mta1 << " and " << mta2 << endl;
            cout << " HELO " << mta1 << "\n"; cout << " 250\n";
            cout << " MAIL FROM:<" << s << ">\n"; cout << " 250\n";
            bool ok = false;
            for(int i = 0; i < users.size(); i++) {
                cout << " RCPT TO:<" << users[i] << ">\n";
                if(addr.count(users[i])) { ok = true; cout << " 250\n"; }
                else cout << " 550\n";
            }
        }
    }
}

```



```

if(ok) {
    cout << " DATA\n"; cout << " 354\n";
    cout << data;
    cout << ".\n"; cout << " 250\n";
}
cout << " QUIT\n"; cout << " 221\n";
}
}
return 0;
}

```

例题 5-12 城市正视图 (Urban Elevations, ACM/ICPC World Finals 1992, UVa221)

如图 5-4 所示, 有 n ($n \leq 100$) 个建筑物。左侧是俯视图 (左上角为建筑物编号, 右下角为高度), 右侧是从南向北看的正视图。

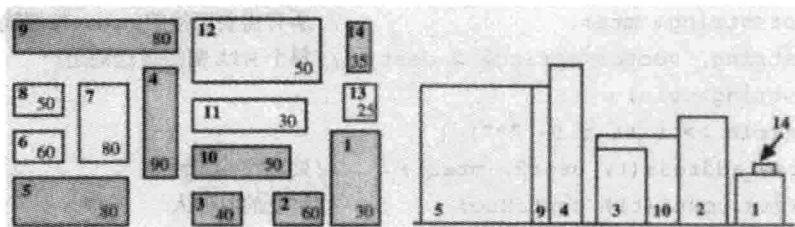


图 5-4 建筑俯视图与正视图

输入每个建筑物左下角坐标 (即 x 、 y 坐标的最小值)、宽度 (即 x 方向的长度)、深度 (即 y 方向的长度) 和高度 (以上数据均为实数), 输出正视图中能看到的所有建筑物, 按照左下角 x 坐标从小到大进行排序。左下角 x 坐标相同时, 按 y 坐标从小到大排序。

输入保证不同的 x 坐标不会很接近 (即任意两个 x 坐标要么完全相同, 要么差别足够大, 不会引起精度问题)。

【分析】

注意到建筑物的可见性等价于南墙的可见性, 可以在输入之后直接忽略“深度”这个参数。接下来把建筑物按照输出顺序排序, 然后依次判断每个建筑物是否可见。

判断可见性看上去比较麻烦, 因为一个建筑物可能只有部分可见, 无法枚举所有 x 坐标, 来查看这个建筑物在该处是否可见, 因为 x 坐标有无穷多个。解决方法有很多种, 最常见的是离散化, 即把无穷变为有限。

具体方法是: 把所有 x 坐标排序去重, 则任意两个相邻 x 坐标形成的区间具有相同属性, 一个区间要么完全可见, 要么完全不可见。这样, 只需在这个区间里任选一个点 (例如中点), 就能判断出一个建筑物是否在整个区间内可见。如何判断一个建筑物是否在某个 x 坐标处可见呢? 首先, 建筑物的坐标中必须包含这个 x 坐标, 其次, 建筑物南边不能有另外一个建筑物也包含这个 x 坐标, 并且不比它矮。

```

#include<cstdio>
#include<algorithm>

```

```

using namespace std;

const int maxn = 100 + 5;

struct Building {
    int id;
    double x, y, w, d, h;
    bool operator < (const Building& rhs) const {
        return x < rhs.x || (x == rhs.x && y < rhs.y);
    }
} b[maxn];

int n;
double x[maxn*2];

bool cover(int i, double mx) {
    return b[i].x <= mx && b[i].x+b[i].w >= mx;
}

//判断建筑物 i 在 x=mx 处是否可见
bool visible(int i, double mx) {
    if(!cover(i, mx)) return false;
    for(int k = 0; k < n; k++)
        if(b[k].y < b[i].y && b[k].h >= b[i].h && cover(k, mx)) return false;
    return true;
}

int main() {
    int kase = 0;
    while(scanf("%d", &n) == 1 && n) {
        for(int i = 0; i < n; i++) {
            scanf("%lf%lf%lf%lf%lf", &b[i].x, &b[i].y, &b[i].w, &b[i].d, &b[i].h);
            x[i*2] = b[i].x; x[i*2+1] = b[i].x + b[i].w;
            b[i].id = i+1;
        }
        sort(b, b+n);
        sort(x, x+n*2);
        int m = unique(x, x+n*2) - x; //x坐标排序后去重, 得到 m 个坐标

        if(kase++) printf("\n");
        printf("For map #%d, the visible buildings are numbered as follows:\n%d",
            kase, b[0].id);
    }
}

```

```

for(int i = 1; i < n; i++) {
    bool vis = false;
    for(int j = 0; j < m-1; j++)
        if(visible(i, (x[j] + x[j+1]) / 2)) { vis = true; break; }
    if(vis) printf(" %d", b[i].id);
}
printf("\n");
}
return 0;
}

```

注意上述代码用到了前面提到的 `unique`。它必须在 `sort` 之后调用，而且 `unique` 本身不会删除元素，而只是把重复元素移到了后面。关于 `unique` 的详细用法请读者自行查阅资料。

5.5 习 题

本章是语言篇的最后一章，介绍了很多可选但是有用的 C++ 语言特性和库函数。有些库函数实际上已经涉及后面要介绍的算法和数据结构，但是在学习原理之前，仍然可以先练习使用这些函数。

如表 5-1 所示是例题列表，其中前 9 道题是必须掌握的。后面 3 题虽然相对比较复杂，但是也强烈建议读者试一试，锻炼编程能力。

表 5-1 例题列表

类 别	题 号	题目名称（英文）	备 注
例题 5-1	UVa10474	Where is the Marble?	排序和查找
例题 5-2	UVa101	The Blocks Problem	vector 的使用
例题 5-3	UVa10815	Andy's First Dictionary	set 的使用
例题 5-4	UVa156	Anagrams	map 的使用
例题 5-5	UVa12096	The SetStack Computer	stack 与 STL 其他容器的综合运用
例题 5-6	UVa540	Team Queue	queue 与 STL 其他容器的综合运用
例题 5-7	UVa136	Ugly Numbers	priority_queue 的使用
例题 5-8	UVa400	Unix ls	排序和字符串处理
例题 5-9	UVa1592	Database	map 的妙用
例题 5-10	UVa207	PGA Tour Prize Money	排序和其他细节处理
例题 5-11	UVa814	The Letter Carrier's Rounds	字符串以及 STL 容器的综合运用
例题 5-12	UVa221	Urban Elevations	离散化

本章的习题主要是为了练习 C++ 语言以及 STL，程序本身并不一定很复杂。建议读者至少完成 8 道习题。如果想达到更好的效果，建议完成 12 题或更多。

习题 5-1 代码对齐 (Alignment of Code, ACM/ICPC NEERC 2010, UVa1593)

输入若干行代码, 要求各列单词的左边界对齐且尽量靠左。单词之间至少要空一格。每个单词不超过 80 个字符, 每行不超过 180 个字符, 一共最多 1000 行, 样例输入与输出如图 5-5 所示。

样例输入	样例输出
start: integer; // begins here	start: integer; // begins here
stop: integer; // ends here	stop: integer; // ends here
s: string;	s: string;
c: char; // temp	c: char; // temp

图 5-5 对齐代码的样例输入与输出

习题 5-2 Ducci 序列 (Ducci Sequence, ACM/ICPC Seoul 2009, UVa1594)

对于一个 n 元组 $(a_1, a_2, \dots, a_n)$, 可以对于每个数求出它和下一个数的差的绝对值, 得到一个新的 n 元组 $(|a_1 - a_2|, |a_2 - a_3|, \dots, |a_n - a_1|)$ 。重复这个过程, 得到的序列称为 Ducci 序列, 例如:

$(8, 11, 2, 7) \rightarrow (3, 9, 5, 1) \rightarrow (6, 4, 4, 2) \rightarrow (2, 0, 2, 4) \rightarrow (2, 2, 2, 2) \rightarrow (0, 0, 0, 0)$ 。

也有的 Ducci 序列最终会循环。输入 n 元组 ($3 \leq n \leq 15$), 你的任务是判断它最终会变成 0 还是会循环。输入保证最多 1000 步就会变成 0 或者循环。

习题 5-3 卡片游戏 (Throwing cards away I, UVa 10935)

桌上有 n ($n \leq 50$) 张牌, 从第一张牌 (即位于顶面的牌) 开始, 从上往下依次编号为 $1 \sim n$ 。当至少还剩下两张牌时进行以下操作: 把第一张牌扔掉, 然后把新的第一张牌放到整叠牌的最后。输入每行包含一个 n , 输出每次扔掉的牌以及最后剩下的牌。

习题 5-4 交换学生 (Foreign Exchange, UVa 10763)

有 n ($1 \leq n \leq 500000$) 个学生想交换到其他学校学习。为了简单起见, 规定每个想从 A 学校换到 B 学校的学生必须找一个想从 B 换到 A 的“搭档”。如果每个人都能找到搭档 (一个人不能当多个人的搭档), 学校就会同意他们交换。每个学生用两个整数 A、B 表示, 你的任务是判断交换是否可以进行。

习题 5-5 复合词 (Compound Words, UVa 10391)

给出一个词典, 找出所有的复合词, 即恰好有两个单词连接而成的单词。输入每行都是一个由小写字母组成的单词。输入已按照字典序从小到大排序, 且不超过 120000 个单词。输出所有复合词, 按照字典序从小到大排列。

习题 5-6 对称轴 (Symmetry, ACM/ICPC Seoul 2004, UVa1595)

给出平面上 N ($N \leq 1000$) 个点, 问是否可以找到一条竖线, 使得所有点左右对称。例如图 5-6 中, 左边的图形有对称轴, 右边没有。

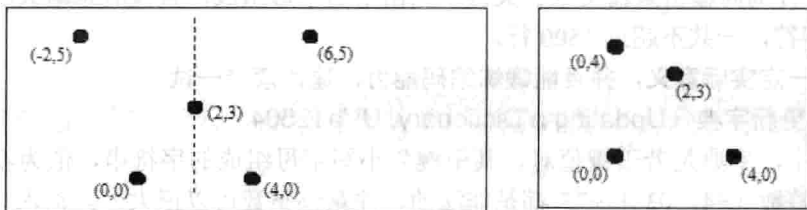


图 5-6 对称轴

习题 5-7 打印队列 (Printer Queue, ACM/ICPC NWERC 2006, UVa12100)

学生会里只有一台打印机,但是有很多文件需要打印,因此打印任务不可避免地需要等待。有些打印任务比较急,有些不那么急,所以每个任务都有一个 1~9 间的优先级,优先级越高表示任务越急。

打印机的运作方式如下:首先从打印队列里取出一个任务 J,如果队列里有比 J 更急的任务,则直接把 J 放到打印队列尾部,否则打印任务 J (此时不会把它放回打印队列)。

输入打印队列中各个任务的优先级以及所关注的任务在队列中的位置(队首位置为 0),输出该任务完成的时刻。所有任务都需要 1 分钟打印。例如,打印队列为 {1, 1, 9, 1, 1, 1}, 目前处于队首的任务最终完成时刻为 5。

习题 5-8 图书管理系统 (Borrowers, ACM/ICPC World Finals 1994, UVa230)

你的任务是模拟一个图书管理系统。首先输入若干图书的标题和作者(标题各不相同,以 END 结束),然后是若干指令: BORROW 指令表示借书, RETURN 指令表示还书, SHELVES 指令表示把所有已归还但还未上架的图书排序后依次插入书架并输出图书标题和插入位置(可能是第一本书或者某本书的后面)。

图书排序的方法是先按作者从小到大排,再按标题从小到大排。在处理第一条指令之前,你应当先将所有图书按照这种方式排序。

习题 5-9 找 bug (Bug Hunt, ACM/ICPC Tokyo 2007, UVa1596)

输入并模拟执行一段程序,输出第一个 bug 所在的行。每行程序有两种可能:

- 数组定义,格式为 `arr[size]`。例如 `a[10]` 或者 `b[5]`,可用下标分别是 0~9 和 0~4。定义之后所有元素均为未初始化状态。
- 赋值语句,格式为 `arr[index]=value`。例如 `a[0]=3` 或者 `a[a[0]]=a[1]`。

赋值语句可能会出现两种 bug: 下标 index 越界;使用未初始化的变量(index 和 value 都可能出现这种情况)。

程序不超过 1000 行,每行不超过 80 个字符且所有常数均为小于 2^{31} 的非负整数。

习题 5-10 在 Web 中搜索 (Searching the Web, ACM/ICPC Beijing 2004, UVa1597)

输入 n 篇文章和 m 个请求 ($n < 100$, $m \leq 50000$),每个请求都是以下 4 种格式之一。

- A: 查找包含关键字 A 的文章。
- A AND B: 查找同时包含关键字 A 和 B 的文章。
- A OR B: 查找包含关键字 A 或 B 的文章。
- NOT A: 查找不包含关键字 A 的文章。

处理询问时,需要对于每篇文章输出证据。前 3 种询问输出所有至少包含一个关键字的行,第 4 种询问输出整篇文章。关键字只由小写字母组成,查找时忽略大小写。每行不超过 80 个字符,一共不超过 1500 行。

本题有一定实际意义,并且能锻炼编码能力,建议读者一试。

习题 5-11 更新字典 (Updating a Dictionary, UVa12504)

在本题中,字典是若干键值对,其中键为小写字母组成的字符串,值为没有前导零或正号的非负整数(-4, 03 和 +77 都是非法的,注意该整数可以很大)。输入一个旧字典和一个新字典,计算二者的变化。输入的两个字典中键都是唯一的,但是排列顺序任意。具

体格式为（注意字典格式中不含任何空白字符）：

```
{key:value,key:value,...,key:value}
```

输入包含两行，各包含不超过 100 个字符，即旧字典和新字典。输出格式如下：

- ❑ 如果至少有一个新增键，打印一个“+”号，然后是所有新增键，按字典序从小到大排列。
- ❑ 如果至少有一个删除键，打印一个“-”号，然后是所有删除键，按字典序从小到大排列。
- ❑ 如果至少有一个修改键，打印一个“*”号，然后是所有修改键，按字典序从小到大排列。
- ❑ 如果没有任何修改，输出 No changes。

例如，若输入两行分别为 {a:3,b:4,c:10,f:6} 和 {a:3,c:5,d:10,ee:4}，输出为以下 3 行：+d,ee;-b,f; *c。

习题 5-12 地图查询 (Do You Know The Way to San Jose?, ACM/ICPC World Finals 1997, UVa511)

有 n 张地图（已知名称和某两个对角线端点的坐标）和 m 个地名（已知名称和坐标），还有 q 个查询。每张地图都是边平行于坐标轴的矩形，比例定义为高度除以宽度的值。每个查询包含一个地名和详细等级 i 。面积相同的地图总是属于同一个详细等级。假定包含此地名的地图中一共有 k 种不同的面积，则合法的详细等级为 $1 \sim k$ （其中 1 最不详细， k 最详细，面积越小越详细）。如果详细等级 i 的地图不止一张，则输出地图中心和查询地名最接近的一张；如果还有并列的，地图长宽比应尽量接近 0.75（这是 Web 浏览器的比例）；如果还有并列，查询地名和地图右下角的坐标应最远（对应最少的滚动条移动）；如果还有并列，则输出 x 坐标最小的一个。如果查询的地名不存在或者没有地图包含它，或者包含它的地图总数超过 i ，应报告查询非法（并输出包含它的最详细地图名称，如果存在）。

提示：本题的要求比较细致，如果打算编程实现，建议参考原题。

习题 5-13 客户中心模拟 (Queue and A, ACM/ICPC World Finals 2000, UVa822)

你的任务是模拟一个客户中心运作情况。客服请求一共有 n ($1 \leq n \leq 20$) 种主题，每种主题用 5 个整数描述：tid, num, t0, t, dt，其中 tid 为主题的唯一标识符，num 为该主题的请求个数，t0 为第一个请求的时刻，t 为处理一个请求的时间，dt 为相邻两个请求之间的间隔（为了简单情况，假定同一个主题的请求按照相同的间隔到达）。

客户中心有 m ($1 \leq m \leq 5$) 个客服，每个客服用至少 3 个整数描述：pid, k, tid₁, tid₂, ..., tid_k，表示一个标识符为 pid 的人可以处理 k 种主题的请求，按照优先级从大到小依次为 tid₁, tid₂, ..., tid_k。当一个人有空时，他会按照优先级顺序找到第一个可以处理的请求。如果有多个同时选中了某个请求，上次开始处理请求的时间早的人优先；如果有并列，id 小的优先。输出最后一个请求处理完毕的时刻。

习题 5-14 交易所 (Exchange, ACM/ICPC NEERC 2006, UVa1598)

你的任务是为交易所设计一个订单处理系统。要求支持以下 3 种指令。

- BUY $p\ q$: 有人想买, 数量为 p , 价格为 q 。
- SELL $p\ q$: 有人想卖, 数量为 p , 价格为 q 。
- CANCEL i : 取消第 i 条指令对应的订单 (输入保证该指令是 BUY 或者 SELL)。

交易规则如下: 对于当前买订单, 若当前最低卖价 (ask price) 低于当前出价, 则发生交易; 对于当前卖订单, 若当前最高买价 (bid price) 高于当前价格, 则发生交易。发生交易时, 按供需物品个数的最小值交易。交易后, 需修改订单的供需物品个数。当出价或价格相同时, 按订单产生的先后顺序发生交易。输入输出细节请参考原题。

提示: 本题是一个不错的优先队列练习题。

习题 5-15 Fibonacci 的复仇 (Revenge of Fibonacci, ACM/ICPC Shanghai 2011, UVa12333)

Fibonacci 数的定义为: $F(0)=F(1)=1$, 然后从 $F(2)$ 开始, $F(i)=F(i-1)+F(i-2)$ 。例如, 前 10 项 Fibonacci 数分别为 1, 1, 2, 3, 5, 8, 13, 21, 34, 55……

有一天晚上, 你梦到了 Fibonacci, 它告诉你一个有趣的 Fibonacci 数。醒来以后, 你只记得了它的开头几个数字。你的任务是找出以它开头的最小 Fibonacci 数的序号。例如以 12 开头的最小 Fibonacci 数是 $F(25)$ 。输入不超过 40 个数字, 输出满足条件的序号。

如果序号小于 100000 的 Fibonacci 数均不满足条件, 输出 -1。

提示: 本题有一定效率要求。如果高精度代码比较慢, 可能会超时。

习题 5-16 医院设备利用 (Use of Hospital Facilities, ACM/ICPC World Finals 1991, UVa212)

医院里有 n ($n \leq 10$) 个手术室和 m ($m \leq 30$) 个恢复室。每个病人首先会被分配到一个手术室, 手术后会分配到一个恢复室。从任意手术室到任意恢复室的时间均为 t_1 , 准备一个手术室和恢复室的时间分别为 t_2 和 t_3 (一开始所有手术室和恢复室均准备好, 只有接待完一个病人之后才需要为下一个病人准备)。

k 名 ($k \leq 100$) 病人按照花名册顺序排队, T 点钟准时开放手术室。每当有准备好的手术室时, 队首病人进入其中编号最小的手术室。手术结束后, 病人应立刻进入编号最小的恢复室。如果有多个病人同时结束手术, 编号较小的病人优先进入编号较小的恢复室。输入保证病人无须排队等待恢复室。

输入 n 、 m 、 T 、 t_1 、 t_2 、 t_3 、 k 和 k 名病人的名字、手术时间和恢复时间, 模拟这个过程。输入输出细节请参考原题。

提示: 虽然是个模拟题, 但是最好先理清思路, 减少不必要的麻烦。本题是一个很好的编程练习, 但难度也不小。